

2025 计算机体系结构-作业 3

学号：2023212290 姓名：朱清扬

1. 动态调度算法 (10 分)

(1)Tomasulo 算法是否能完全消除 RAW、WAR 和 WAW 相关,如果能,请简要说明实现机制,如果不能请说明理由。(3 分)

答：

Tomasulo 算法无法完全消除所有相关。

对于反相关 (WAR) 和输出相关 (WAW), Tomasulo 算法通过使用寄存器重命名 (将指令中的寄存器动态地换成数值或指向保留站的指针) 的机制,能够动态消除这些名相关,从而减少流水线冒险造成的停顿。

然而,对于真相关 (RAW),它是指令间基于值的依赖关系,是必须一直遵从的,无法被消除。Tomasulo 算法通过让等待操作数的指令监测公共数据总线 (CDB),将结果直接从产生者指令传输给消费者指令 (数据前推) 来处理 RAW 冲突,以减少停顿,但这种数据前推无法消除所有 RAW 相关造成的流水线冒险。

(2) 请解释什么叫精确异常? (3 分)

答：

精确异常是指当指令 导致发生异常时,处理机的现场 (状态) 与严格按照程序顺序执行指令 发生异常时的现场 (状态) 相同。

这意味着在发生异常时,所有按程序顺序位于指令 之前的指令必须已经完成,并且更新了处理器状态 (如寄存器和内存),而所有按程序顺序位于指令 之后的指令,必须表现得像从未开始执行一样,没有改变任何处理器状态。

(3) Tomasulo 算法能否实现精确异常?为什么? (4 分)

答：

Tomasulo 算法本身是动态调度的一种，其指令执行是按序发送、乱序执行、乱序完成的，指令乱序完成会增加异常处理的难度，并可能发生不精确异常。

要实现精确异常，动态调度处理机必须保持正确的异常行为。Tomasulo 算法可以通过对其进行扩充来支持前瞻执行 (speculative execution)。前瞻执行的关键思想是允许指令乱序执行，但必须顺序确认 (commit)。通过这种扩充，将写结果与指令确认区分开来，并通过重排序缓冲器 (ROB) 实现指令的顺序完成和确认，从而能够实现精确异常。

因此，扩展后的 Tomasulo 算法 (支持前瞻执行) 可以实现精确异常。

2. 性能分析 (6 分)

假设有一个长流水线，仅仅对条件转移指令使用分支目标缓冲 (BTB)。此外，假设分支预测错误的开销为 4 个时钟周期，缓存缺失的开销为 3 个时钟周期，缓存命中率为 90%，分支预测精度为 90%，分支指令频率为 15%，没有分支的基本 CPI 为 1。根据这些条件，计算：

(1) 求程序执行的 CPI。(3 分)

(2) 相对于采用固定的 2 个时钟周期延迟的分支处理，哪种方法程序执行速度更快？(3 分)

答：先算由于分支带来的平均额外周期数/条指令：

分支频率 ($f_b=15\%=0.15$)

BTB 命中率 ($h=90\%=0.9$)，未命中开销 $P_{miss}=3$ 周期

预测精度 ($a=90\%=0.9$)，误判概率 ($1-a=0.1$)，误判开销 $P_{misp} = 4$ 周期

对一条分支，期望开销：

$$E = h \cdot (1-a) \cdot P_{\text{misp}} + (1-h) \cdot P_{\text{miss}} = 0.9 \cdot 0.1 \cdot 4 + 0.1 \cdot 3 = 0.36 + 0.3 = 0.66$$

折算到每条指令的平均开销： $0.15 \times 0.66 = 0.099$ 。

因此程序的 CPI： $\text{CPI} = \text{CPI}_0 + 0.099 = 1 + 0.099 \approx 1.10$

(2) 固定“2 个时钟周期延迟”的分支处理：每条分支必付出 2 周期：

$$\text{CPI} = 1 + 0.15 \times 2 = 1.30$$

比较可知，采用“BTB+分支预测”的方法更快。其相对加速比

$$1.30 / 1.099 \approx 1.18$$

约快 18%。

3. 相关与冒险 (6 分)

请阐述 MIPS 的 5 段流水线如何检测数据相关所造成的冒险？(4 分)

答：

1. 记分板检测 (Scoreboard)：为寄存器文件中每个寄存器配置有效位 (valid bit)。处于译码阶段的指令检查其所需的源和目的寄存器的有效位是否为有效 (valid)。如果无效，则存在数据相关，需要暂停该指令的处理。
2. 组合逻辑检测：使用专门的检测逻辑来检测当前处于后续流水段中的指令的目标寄存器是否是处于译码阶段的指令的源寄存器。如果是，则暂停译码段中指令的处理。

在 MIPS 5 级流水线中，真相关 (RAW) 指令 IA 和 IB (IA 在 IB 之前) 导致流水线冒险，当且仅当：IB (R/I, LW, SW, Br or JR 等类型指令) 读了一个被 IA (R/I or LW 类型指令) 写入的寄存器，且两条指令间的距离 $\text{dist}(IA, IB) \leq 3$ 。

检测逻辑需要判断处于 ID 段的指令是否试图读取一个尚未被前序指令写

入结果的寄存器：

当处于 ID 段的指令 (IB) 想读取一个处于 EX、MEM 或者 WB 段的指令 (IA) 要写入的目标寄存器时，流水线必须暂停 ID 段的处理。

暂停条件的检测逻辑需要检查处于 ID 段的指令所需的源寄存器 (rs 或 rt) 是否与处于后续流水段 (EX, MEM, WB) 的指令的目标寄存器 (dest) 一致，并且后续指令具有寄存器写使能。

为什么名相关对于顺序执行的 5 段 MIPS 流水线来说不会造成冒险 ? (2 分)

答：

这是因为反相关和输出相关相对容易处理，通常可以通过在流水线的某一阶段按照程序顺序来写目标寄存器即可解决。在标准的 MIPS 5 段流水线中，寄存器文件的写入操作 (Write Back, WB) 是流水线最后一个阶段，指令是按程序顺序流出和完成写入的，这天然保持了 WAR 和 WAW 所要求的顺序，因此不会导致冒险。

4. 缓存一致性协议 (8 分)

请对比分析缓存一致性协议中写更新和写无效两种方案的优缺点。(4 分)

答：

写更新和写无效是缓存一致性协议的两种实现方案：

1. 写更新 (Write Update)：

机制：当核心要写入一个块时，需通过总线广播要写入的数据。

所有侦听缓存会通过总线更新其副本。

缺点： 看起来最简单、直接、速度快，但会消耗大量网络带宽。

对同一个字或同一个缓存块的多次写操作，在写更新下每个写都需要一次更新（更多的流量），因此写更新的开销更大。

2. 写无效 (Write Invalidate)：

机制： 当核心要写入一个块时，会发送 “write invalidate” 消息。

所有侦听缓存会使自己的副本失效 (invalidate)。

优点： 经验表明，写无效机制会使用相当少的总线带宽资源。对同一个字或同一个缓存块的多个写操作，只需要一个 “write invalidate” 消息。

应用： 由于总线带宽在多核系统中是稀缺资源，当前大多数产品采取写无效方案。

请分析目录一致性协议的优点，和存在的问题。(4 分)

答：

优点 (优势)：

具有更好的可扩展性。

通过记录一个内存块(memory block)的共享者信息来避免广播，

因此可以使用点到点的消息来维持一致性。

更加灵活，可以适用于任何可扩展的点到点网络拓扑。

存在的问题 (挑战):

内存和目录带宽的扩展性问题：不能使用集中的主存 (main

memory) 或者目录 (directory)，需要分布式存储器和目录结构。

目录存储器需求难以很好地扩展：存在的比特数 (presence bits)

随着处理器数量的增加而增长。